

MEMORIA DEL PROYECTO: NEXSHOP

Análisis detallado e implementación de base de datos relacional para NexShop Group S.A.

AUTOR: Rafael García Moreno

FECHA: Junio 2026

1. Caso de Negocio y Objetivos

NexShop Group S.A. es una empresa dedicada a la venta al por menor fundada en 2015 con sede en Valencia. A nivel de negocio, la compañía cuenta con una tienda online (nexshop.es) y tres tiendas físicas físicas en Valencia, Madrid y Barcelona. Tradicionalmente, ambas líneas de venta operaban como silos de información desconectados, lo que generaba serias ineficiencias de stock, pérdida de trazabilidad en las promociones y problemas de atención al cliente al no poder cruzar compras físicas con cuentas online.

Los objetivos principales de este proyecto son:

1. Diseñar un único repositorio de datos relacional que unifique las operaciones de venta física y online.
2. Proveer al departamento de Logística de herramientas para controlar el stock real en cada sede y gestionar transferencias internas.
3. Permitir a Dirección y Marketing evaluar el impacto de las campañas mediante un histórico de precios y un programa de puntos robusto.
4. Facilitar la labor de Atención al Cliente ofreciendo un histórico consolidado de incidencias y devoluciones.

2. Análisis de Requerimientos

El diseño de la base de datos se ha realizado basándose en las actas de reuniones y correos electrónicos cruzados con los responsables de cada una de las áreas clave:

- Logística (David Cano): Solicitó que cada punto de venta y almacén central tuviera niveles de stock específicos para evitar roturas de stock en pedidos online. Además, requería dejar constancia de qué empleado autorizaba cada transferencia de inventario.
- Atención al Cliente (Laura Pons): Puso foco en la necesidad de gestionar tickets de incidencia de manera centralizada y dar soporte a las devoluciones en tienda física. También planteó la problemática de los clientes anónimos que compran presencialmente y más adelante desean registrarse online y heredar sus compras antiguas.
- Operaciones y Márketing (Ana Ferrer): Incidió en que las promociones fuesen temporales y basadas en porcentajes de descuento, y que el catálogo estuviese jerarquizado en categorías y subcategorías. Asimismo, remarcó la importancia de llevar un programa de puntos detallado por transacción sin duplicar saldos estáticos.

3. Normalización y Decisiones de Diseño

Para garantizar la integridad y evitar anomalías de inserción, actualización y borrado, el diseño se ha estructurado en Tercera Forma Normal (3FN). Como ejemplo de esto, se ha evitado almacenar campos calculados como el 'saldo_actual' del cliente o subtotales en las cabeceras de venta que puedan deducirse directamente de las líneas de detalle.

Las tablas intermedias creadas para resolver relaciones N:M (tales como proveedor_producto, linea_pedido, linea_venta y promocion_producto) garantizan que no existan atributos multivalor ni duplicidades. El uso de claves foráneas con políticas de borrado y actualización controladas (como ON DELETE CASCADE o ON DELETE SET NULL) previene registros huérfanos en cascada.

4. Entidades del Sistema (22 Tablas)

sede	Representa tanto a las tiendas físicas como al almacén central. He decidido unificarlas en una tabla con un campo 'tipo' para que sea más fácil gestionar el stock y los traspasos de material entre ellas.	sede_id, nombre, tipo (tienda/almacen), direccion, ciudad, pais, telefono, email
empleado	Los trabajadores de las tiendas y oficinas. Guardamos sus datos personales, fecha de entrada y la sede donde trabajan.	empleado_id, nombre, apellidos, dni, email_corporativo, fecha_incorporacion, rol, sede_id
categoria	Para organizar los productos del catálogo. He puesto una relación con ella misma para poder tener subcategorías de forma flexible.	categoria_id, nombre, descripcion, categoria_padre_id
producto	Las referencias de venta. Cada producto pertenece a una sola categoría.	producto_id, referencia, nombre, descripcion, pvp_actual, activo, categoria_id
historico_precio	Guarda los precios que ha tenido cada producto a lo largo del tiempo con fechas de inicio y fin, para no perder el histórico al actualizar el PVP del producto.	historico_precio_id, producto_id, precio, fecha_inicio, fecha_fin
proveedor	Las empresas que nos suministran los artículos. Cada una tiene asignado un empleado comercial de nuestra empresa como	proveedor_id, nombre_empresa, nif, email, telefono, pais, empleado_comercial_id

	contacto.	
proveedor_producto	Relación de qué proveedores nos venden qué productos. Guardamos las condiciones pactadas (precio y días que tardan en entregarlo).	proveedor_producto_id, proveedor_id, producto_id, precio_coste, plazo_entrega_dias, fecha_inicio, fecha_fin
promocion	Campañas de descuento con fechas concretas de validez.	promocion_id, nombre, descripcion, descuento_porcentual, fecha_inicio, fecha_fin
promocion_producto	Relación N:M para saber qué productos están metidos en cada promoción.	promocion_id, producto_id
cliente	Los clientes registrados en la web o identificados en tienda.	cliente_id, nombre, apellidos, email, password_hash, fecha_nacimiento, telefono, fecha_registro, activo
direccion	Direcciones de envío de los clientes. Un cliente puede tener varias guardadas.	direccion_id, cliente_id, alias, calle, numero, piso, codigo_postal, ciudad, pais, es_principal
pedido_online	Pedidos que se hacen a través de la página web.	pedido_id, cliente_id, direccion_id, fecha_pedido, estado, total, puntos_canjeados, notas
linea_pedido	Los productos detallados y las cantidades dentro de cada pedido online.	linea_pedido_id, pedido_id, producto_id, cantidad, precio_unitario, descuento_aplicado
envio	Los paquetes físicos enviados al cliente. Como un pedido se puede mandar a trozos desde distintas sedes, un pedido puede tener varios envíos.	envio_id, pedido_id, sede_origen_id, numero_seguimiento, transportista, fecha_estimada_entrega,

		fecha_entrega_real, estado
venta_presencial	Ventas que se hacen directamente en caja en las tiendas físicas. Si el cliente no quiere registrarse, el campo de cliente puede ser NULL.	venta_id, sede_id, empleado_id, cliente_id, cliente_anonimo_ref, fecha_venta, total
linea_venta	Detalle de los artículos de los tickets de tienda física.	linea_venta_id, venta_id, producto_id, cantidad, precio_unitario
devolucion	Documentos de devolución de artículos comprados presencialmente.	devolucion_id, venta_id, producto_id, cantidad, motivo, fecha_devolucion, importe_reembolsado
ticket_incidencia	Quejas o consultas de los clientes que atienden los empleados de soporte.	ticket_id, cliente_id, empleado_id, pedido_id, asunto, descripcion, fecha_apertura, estado, fecha_cierre, nota_resolucion
stock	El inventario de cada producto en cada una de las tiendas o almacenes.	stock_id, sede_id, producto_id, cantidad
transferencia_stock	Movimientos de mercancía de una tienda a otra autorizados por un empleado.	transferencia_id, producto_id, sede_origen_id, sede_destino_id, cantidad, fecha, empleado_id, estado
valoracion	Reseñas que ponen los clientes a los productos.	valoracion_id, cliente_id, producto_id, puntuacion, comentario, fecha, es_verificada, pedido_id
movimiento_puntos	Historial de los puntos de fidelidad que gana o gasta un cliente.	movimiento_id, cliente_id, tipo, puntos, pedido_id, fecha, descripcion

5. Preguntas de reflexión del proyecto

Pregunta 1: Un pedido online puede generar varios envíos. ¿Cómo lo has modelado y por qué?

Lo he resuelto creando una tabla específica llamada 'envio'. Esta tabla tiene una clave ajena (pedido_id) que apunta al pedido online original. De esta forma, si para un solo pedido tenemos que mandar algunas cosas desde el almacén de Valencia y otras desde la tienda de Madrid porque no hay stock de todo en el mismo sitio, podemos registrar dos envíos diferentes para ese único pedido. Cada envío tiene su propio transportista y número de seguimiento para que el cliente pueda ver dónde está cada paquete.

Pregunta 2: ¿Una sola tabla o dos para ventas presenciales y pedidos online?

He decidido usar dos tablas separadas: 'pedido_online' y 'venta_presencial'.

¿Por qué? Porque son procesos muy distintos. Los pedidos online necesitan una dirección de envío, generan paquetes con transportistas y pasan por varios estados (enviado, entregado). En cambio, la venta presencial se hace en el momento en una caja física por un vendedor, no necesita dirección y el ticket es definitivo. Si usáramos una sola tabla, tendríamos un montón de columnas vacías (NULL) dependiendo de cómo se haga la compra, lo cual me parece un diseño un poco sucio.

Pregunta 3: ¿Cómo diferencias el histórico de precios del histórico de promociones?

Los separo porque son cosas totalmente diferentes:

- El histórico de precios (tabla 'historico_precio') registra los cambios que sufre el precio normal de venta (PVP) del producto por temas de mercado o costes.
- Las promociones (tablas 'promocion' y 'promocion_producto') son campañas de marketing temporales (como las rebajas de verano o el Black Friday) donde se aplica un descuento de forma temporal a un grupo de artículos. Así, el precio base sigue siendo el mismo y lo que cambia es el descuento aplicado en ese rango de fechas.

Pregunta 4: Saldo de puntos calculable siempre desde histórico. ¿Guardas saldo_actual?

No he puesto un campo 'saldo_actual' en la tabla de clientes. Como en las reuniones nos dijeron que el saldo se tiene que poder calcular siempre desde el histórico de movimientos, prefiero calcularlo al vuelo con una consulta SQL sumando los puntos ganados y restando los canjeados de la tabla 'movimiento_puntos'. Así evitamos que si por algún fallo se modifica el historial de puntos, el saldo de la ficha de cliente se quede desactualizado o descuadrado.

Pregunta 5: Cliente presencial que luego se registra online. ¿Cómo lo has previsto?

En la tabla 'venta_presencial' he puesto el campo 'cliente_id' como nullable (puede ser NULL) para las personas que compran en tienda de forma anónima sin dar sus datos. Pero he añadido un campo de texto llamado 'cliente_anonimo_ref' donde el cajero puede apuntar el nombre o el teléfono si el cliente quiere. Si más adelante ese cliente decide crearse una cuenta online y nos pide asociar sus compras antiguas, podemos buscar esos tickets viejos usando la referencia y actualizar el 'cliente_id' con su nueva ID.

Pregunta 6: ¿Has decidido NO implementar o simplificar algún requisito?

He decidido no meter una tabla de notificaciones para los envíos de promociones o cumpleaños, porque me parece que eso es algo que debería manejar la aplicación externa o el backend programando un script (por ejemplo, buscando qué clientes cumplen años hoy con la columna fecha_nacimiento de la base de datos). También he hecho un flujo de estados para los traspasos de stock muy básico con un ENUM simple para no complicar el diseño con tablas de eventos innecesarias.